

Learn Low-Level Design in 25 Days: Day 4— Composition vs Inheritance: The Eternal Trade-off

(Part of the series: Low-Level Design in 25 Days—A Practical Bootcamp for Devs)

TechPreneur

Published October 17, 2025

Free: No

Read on Freedium · original

Contents

1. When Inheritance Became a Headache 0

2. The Core Concept: Two Ways to Reuse Code 0

Inheritance — The "Is-A" Relationship 0

Composition — The "Has-A" Relationship 0

3. Why Composition Often Wins 0

4. Real-World Example: Slack's Notification System 0

5. When to Use Which 0

6. Try It Yourself 0

7. Summary & Next Step 0

1. When Inheritance Became a Headache

A few years ago, I joined a team maintaining a legacy Java codebase — the kind where class names looked like `SuperBaseManagerV2`. Everything inherited from everything. Need to add logging? Extend `BaseService`. Need caching? Extend `CacheableService`. Need both? Well, good luck — Java doesn't do multiple inheritance.

Within a few months, the team realized something painful: **inheritance made our design rigid**. A single change in a parent class could ripple across dozens of child classes, breaking things we never intended to touch.



Welcome to **Day 4 of "Learn Low-Level Design in 25 Days."** Yesterday, we explored Interfaces, Abstract Classes, and Inheritance. Today, we'll take that a step further — learning when not to inherit and why **composition** often leads to cleaner, more flexible systems.

By the end of this article, you'll know **how to decide between inheritance and composition**, and how companies like Uber and Slack structure their systems around these trade-offs.

2. The Core Concept: Two Ways to Reuse Code

Inheritance — The "Is-A" Relationship

Inheritance is about hierarchy. A Dog **is a** Animal. A SportsCar **is a** Car.

It's straightforward — you define common behavior in a base class, and child classes reuse or override it.

```
class Car {
    void startEngine() {
        System.out.println("Starting engine...");
    }
}

class SportsCar extends Car {
    void turboBoost() {
        System.out.println("Activating turbo!");
    }
}
```

Simple, right? But here's where trouble starts — what if you also want ElectricCar? And SelfDrivingCar? You'll soon have a **fragile hierarchy** with overlapping responsibilities and duplicated code.

Inheritance gives **tight coupling** — subclasses depend on the implementation details of the parent. Change the parent, and everything downstream breaks.

Composition — The "Has-A" Relationship

Composition is about **assembling** behavior instead of inheriting it.

A Car **has an** Engine. A Car **has a** GPSModule. A Car **has a** TurboSystem.

You combine small, independent components instead of baking everything into a class hierarchy.

```
class Engine {
    void start() {
        System.out.println("Engine started");
    }
}

class TurboSystem {
    void boost() {
        System.out.println("Turbo engaged!");
    }
}

class Car {
    private Engine engine;
    private TurboSystem turbo;

    Car(Engine engine, TurboSystem turbo) {
        this.engine = engine;
        this.turbo = turbo;
    }

    void drive() {
        engine.start();
        turbo.boost();
    }
}
```

Now, want an ElectricCar? Just plug in a different engine:

```
class ElectricEngine extends Engine {
    void start() {
        System.out.println("Silently starting electric engine...");
    }
}
```

No hierarchy hell. No broken dependencies. Just swappable parts.

3. Why Composition Often Wins

Real-world systems evolve. You rarely know all possible features when you start — and inheritance locks you into early assumptions.

Composition keeps your system open for extension, closed for modification — echoing the Open/Closed Principle from Day 2.

Let's visualize it:

```
Inheritance:
BaseClass
   |
   +-- SubClass1
   |
   +-- SubClass2
        |
        +-- SubSubClass (and so on...)

Composition:
Car
 ├── Engine
 ├── TurboSystem
 └── GPSModule
```

Inheritance gives you a **tree** (rigid and vertical). Composition gives you a **graph** (modular and flexible).

4. Real-World Example: Slack's Notification System

Slack's notification system supports multiple channels — desktop, mobile, and email.

An inheritance-based approach might look like:

```
Notification
   ├── EmailNotification
   ├── MobileNotification
   └── DesktopNotification
```

But what happens when Slack adds a new priority filter or sound settings feature? Each subclass needs to change — violating the Open/Closed Principle.

Instead, Slack engineers use composition:

```
class Notification {
    private Channel channel;
    private AlertTone tone;

    Notification(Channel channel, AlertTone tone) {
        this.channel = channel;
        this.tone = tone;
    }

    void send(String message) {
        tone.play();
        channel.deliver(message);
    }
}
```

Now you can plug in:

- `EmailChannel` or `PushChannel`
- `SilentTone` or `LoudTone`

No class explosion. No messy overrides. Just **composable, testable pieces**.

5. When to Use Which

Use Inheritance When:

- There's a clear "is-a" relationship.
- Behavior truly needs to be shared across all subclasses.
- You don't expect frequent changes to the base class.

Use Composition When:

- You want to combine flexible, reusable parts.
- Behavior might change or vary dynamically.
- You want to follow SOLID principles and keep coupling low.

Think of inheritance as **static reuse**, and composition as **dynamic assembly**.

6. Try It Yourself

Challenge #1: Design a Payment system for an e-commerce app. You have multiple payment gateways (Razorpay, Stripe, PayPal). Now, you need to add discount calculation and logging. Would you use inheritance or composition? Sketch a quick class diagram.

Challenge #2: Look at your current project. Find one place where inheritance made code brittle. Can you refactor it using composition?

7. Summary & Next Step

Key Takeaways

- Inheritance = "is-a" (hierarchy, reuse through extension)
- Composition = "has-a" (flexibility, reuse through delegation)
- Inheritance is great for stable, shared behavior — but avoid deep hierarchies
- Composition promotes testability, modularity, and change tolerance
- Most modern architectures (like microservices) favor composition

Tomorrow, we'll explore **"How to Think in Objects: Turning Requirements into Classes"** — where we'll take real-world use cases and turn them into clean, modular designs step-by-step.

You can follow and save the full series here: Low-Level Design in 25 Days — A Practical Bootcamp for Devs (https://freedium-mirror.cfd/https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8)